

Python Collectables

There are four collection data types in the Python programming language:

- **Set:** is unordered, unchangeable*, and unindexed. No duplicate members.
- **List:** is ordered and changeable. Allows duplicate members.
- **Tuple:** is ordered and unchangeable. Allows duplicate members.
- **Dictionary:** is ordered** and changeable. No duplicate members.

*Set *items* are unchangeable, but you can remove and/or add items whenever you like.

**As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.

Python Collectables - Sets

- **Sets in Python are unordered collections of unique elements.**
- **While sets themselves are mutable, meaning you can add or remove items, the elements within a set must be immutable.**
- **Once an element is added to a set, it cannot be changed.**
- **Sets are unordered, which means the elements do not have a specific index and cannot be accessed by their position.**
- **Duplicate elements are automatically removed when a set is created.**

Python Collectables - Sets

- To create a set, you can use curly braces {} or the set() constructor.

Python



```
my_set = {1, 2, 2, 3}
print(my_set)  # Output: {1, 2, 3}

my_set.add(4)
print(my_set)  # Output: {1, 2, 3, 4}

my_set.remove(2)
print(my_set)  # Output: {1, 3, 4}
```

Python Collectables - Sets

- While you can add or remove items from a set, you cannot change the value of an existing item directly.
- If you need to "change" an item, you would remove the old item and add the new one.

Python



```
my_set.remove(3)
my_set.add(5)
print(my_set) # Output: {1, 4, 5}
```

Python Collectables - Sets

- **If you need a set-like object that is truly immutable, you can use a frozenset.**
- **Once a frozenset is created, it cannot be changed.**

Python



```
frozen_set = frozenset([1, 2, 3])  
# frozen_set.add(4) # This would raise an error
```

Python Collectables - Lists

- **In Python, a list is a versatile and mutable data structure used to store an ordered collection of items.**
- **Lists are defined by enclosing a comma-separated sequence of items within square brackets `[]`.**
- **These items can be of different data types, including numbers, strings, booleans, and even other lists (nested lists).**
- **Lists are ordered, meaning that the items in a list maintain their order of insertion.**
- **This allows you to access items in a list by their index, starting from 0 for the first item.**
- **Lists are also mutable, which means that you can modify their contents after they are created by adding, removing, or changing items.**

Python Collectables - Lists

Creating Lists

Lists are created using square brackets `[]` and can contain elements of different data types.

Python



```
my_list = [1, "hello", 3.14, True]
```

Python Collectables - Lists

Accessing Elements

Elements in a list are accessed using their index, starting from 0.

Python



```
print(my_list[0]) # Output: 1  
print(my_list[1]) # Output: hello
```

Python Collectables - Lists

Modifying Lists

Lists are mutable, meaning their elements can be changed after creation.

Python



```
my_list[2] = "world"  
print(my_list) # Output: [1, "hello", "world", True]
```

Python Collectables - Lists

Adding Elements

- `append()` : Adds an element to the end of the list.
- `insert()` : Inserts an element at a specific position.
- `extend()` : Adds multiple elements from another list or iterable to the end.

```
my_list.append(10)
print(my_list)  # Output: [1, "hello", "world", True, 10]

my_list.insert(1, "new")
print(my_list)  # Output: [1, "new", "hello", "world", True, 10]

my_list.extend([11, 12])
print(my_list)  # Output: [1, "new", "hello", "world", True, 10, 11, 12]
```

Python Collectables - Lists

Removing Elements

- `remove()`: Removes the first occurrence of a specific value.
- `pop()`: Removes the element at a specific index (or the last element if no index is given) and returns it.
- `del`: Removes element at a specific index or slices.

```
my_list.remove("hello")
print(my_list) # Output: [1, "new", "world", True, 10, 11, 12]

popped_element = my_list.pop(1)
print(my_list) # Output: [1, "world", True, 10, 11, 12]
print(popped_element) # Output: new

del my_list[0]
print(my_list) # Output: ['world', True, 10, 11, 12]
```

Python Collectables - Lists

Other Operations

- `len()`: Returns the length of the list.
- `index()`: Returns the index of the first occurrence of a value.
- `count()`: Returns the number of times a value appears in the list.
- `sort()`: Sorts the list in place.
- `reverse()`: Reverses the list in place.
- `copy()`: Returns a shallow copy of the list.

Python Collectables - Lists

Python



```
print(len(my_list)) # Output: 5
print(my_list.index(10)) # Output: 2
print(my_list.count(10)) # Output: 1

my_list.sort(reverse=True)
print(my_list) # Output: [12, 11, 10, True, 'world']

my_list.reverse()
print(my_list) # Output: ['world', True, 10, 11, 12]

new_list = my_list.copy()
print(new_list) # Output: ['world', True, 10, 11, 12]
```

Python Collectables - Lists

List Comprehension

List comprehensions provide a concise way to create new lists based on existing iterables.

Python



```
squares = [x**2 for x in range(10)]  
print(squares) # Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Exercises

Python Collectables

Exercise 1: List Operations (Warm-up)

Task: Create a list of fruits

```
fruits = ["apple", "banana", "mango", "banana", "orange"]
```

a) Print the first and last fruit

b) Add "grape" to the list

c) Remove "banana"

d) Check if "apple" is in the list

e) Print the length of the list

Exercises

Python Collectables

Exercise 2: Set Operations

Task: Create a set of programming languages

```
languages = {"Python", "Java", "C++", "Python", "Ruby"}
```

a) Add "Go" to the set

b) Remove "Java"

c) Check if "C++" is in the set

d) Try to add "Python" again and observe the result

e) Print the set

Exercises

Python Collectables

Exercise 3: Convert List to Set

Task: Remove duplicates from a list using a set

```
numbers = [2, 3, 5, 2, 3, 7, 11, 7]
```

a) Convert to a set

b) Convert back to a list

c) Print the list of unique numbers

Exercises

Python Collectables

Exercise 4: Set Operations – Union, Intersection, Difference

Task: Work with two sets of students

```
math_students = {"Alice", "Bob", "Charlie", "David"}
```

```
cs_students = {"Charlie", "Eve", "Frank", "Alice"}
```

a) Students taking both Math and CS (intersection)

b) Students taking either Math or CS (union)

c) Students in Math but not in CS (difference)

d) Students in only one course (symmetric difference)

Exercises

Python Collectables

Exercise 5: Challenge – Unique Words in a Sentence

Task: Ask user to enter a sentence and return a list of unique words

```
sentence = input("Enter a sentence: ")
```

a) Split the sentence into words

b) Convert to a set to remove duplicates

c) Convert back to a sorted list

d) Print the list of unique words